Compact Sigma Protocols for Standard EC-ElGamal in Confidential Multi-Purpose Tokens on XRPL

Tess Dore Murat Cenk
RippleX Research

May 12, 2026

Abstract

The Confidential Multi-Purpose Token (CMPT) protocol on the XRP Ledger [CMA26] uses standard EC-ElGamal encryption with three separate sigma protocols—same-plaintext equality, amount linkage (Variant A), and balance linkage (Variant B)—contributing approximately 586 bytes to each ConfidentialMPTSend transaction. We show that AND-composing these three protocols under a shared Fiat–Shamir challenge [CDS94] and transmitting the proof in *compact form* (challenge plus response scalars, rather than first-round commitments) reduces the sigma-proof overhead to **192 bytes**: one 32-byte challenge and five 32-byte responses. The verifier reconstructs all commitments algebraically and recomputes the hash. Combined with the shared-C_1 ciphertext layout, this reduces the total ConfidentialMPTSend transaction to **1177** bytes for $n = 4$ recipients (the production configuration: sender, receiver, issuer, auditor), or 1144 bytes for $n = 3$—a 27–29% reduction from the 1604-byte baseline, with no change to security assumptions. We provide full security proofs (completeness, 2-special soundness, HVZK) and implementation guidance (deterministic nonces, Fiat–Shamir domain separation, batch verification tradeoffs). We also present a constant-size (64-byte) proof of shared-randomness plaintext equality across n ciphertexts, due to Cenk [Cen26], that may be useful for multi-auditor extensions. In Section A, we evaluate the twisted ElGamal variant and show that, under compact form, its advantage over standard EC-ElGamal shrinks to an irreducible 64 bytes—insufficient to justify the specification, implementation, and audit costs.

Contents

1 Introduction

The CMPT specification [CMA26] defines a ConfidentialMPTSend transaction whose zero-knowledge proof bundle consists of three components:

(i) An aggregated Bulletproof [BBB+18] range proof on the transfer amount and remainder balance (754 bytes).

(ii) A same-plaintext equality proof Π_{equal} that all ciphertexts encrypt the same amount (229 bytes).

(iii) Two linkage proofs: Variant A (amount commitment to ciphertext, 195 bytes) and Variant B (balance commitment to balance ciphertext, 195 bytes).

Components (ii) and (iii) are standard Sigma protocols made non-interactive via Fiat–Shamir. They share overlapping witnesses—the transfer amount m, shared randomness r, and sender secret key sk_A appear in multiple relations—but the specification treats them as independent proofs with separate challenges and separate first-round commitments.

We observe two compression opportunities that, combined, reduce the sigma-proof overhead from 586 to 192 bytes:

AND-composition.
All three sigma protocols prove statements about the same underlying witness. Merging them under a single Fiat–Shamir challenge [CDS94] eliminates duplicate nonces and responses for shared witnesses, reducing the merged proof to $n + 5$ group-element commitments and 5 scalar responses.

Compact (challenge) form.
In a Fiat–Shamir Sigma protocol, the verifier can *reconstruct* the first-round commitments from the challenge and responses via the verification equations, then recompute the hash to check consistency. This is standard folklore—Schnorr signatures [Sch91] transmit (e, z) rather than (R, z). Applying this to the AND-composed protocol eliminates all $n + 5$ group elements from the proof, leaving only 6 scalars: one challenge and five responses.

The resulting compact proof is $6 \times 32 = 192$ bytes, regardless of the number of recipients n. Combined with the shared-C_1 ciphertext deduplication, the total ConfidentialMPTSend size decreases from 1604 to 1177 bytes for $n = 4$ (sender, receiver, issuer, auditor—the production configuration) or 1144 bytes for $n = 3$.

Scope and structure. Section 2 derives the combined proof statement from the CMPT specification. Section 3 presents the AND-composed sigma protocol in both interactive and compact forms. Section 4 provides full security proofs. Section 5 presents a constant-size shared-randomness equality proof due to Cenk [Cen26]. Section 6 discusses implementation considerations. Section 7 gives the transaction size analysis. Section A evaluates the twisted ElGamal variant and shows that compact form reduces its advantage to 64 bytes.

2 Combined Proof Statement

We derive the combined relation from the CMPT specification, following [Cen26].

2.1 Setup and Notation

Let $\mathbb{G}$ be the secp256k1 group of prime order q with base point G. Let $H \in \mathbb{G}$ be an independent generator (NUMS point) for Pedersen commitments, with $\log_G H$ unknown. Let $m \in \mathbb{Z}_q$ denote the transfer amount and $r \in \mathbb{Z}_q$ the shared encryption randomness. For n public keys $P_1, \ldots, P_n \in \mathbb{G}$—where P_A denotes the sender's public key—the transfer amount is represented by a shared-randomness EC-ElGamal ciphertext family. In production, $n = 4$ (sender, receiver, issuer, auditor); we give sizes for both $n = 3$ and $n = 4$ throughout.

$$C_1 = r \cdot G, \qquad C_{2,i} = m \cdot G + r \cdot P_i, \quad i = 1, \ldots, n.$$

The shared first component C_1 is transmitted once on-chain.

2.2 Pedersen Commitments

The same transfer amount is committed using a Pedersen commitment that intentionally reuses the ciphertext randomness r:

$$PC_m = m \cdot G + r \cdot H.$$

This eliminates the need for a separate Variant A linkage witness: the sigma proof naturally handles the linkage because m and r appear in both the ciphertext and commitment relations.

The sender's balance before transfer is $b \in \mathbb{Z}_q$, committed with fresh randomness $\rho \in \mathbb{Z}_q$:

$$PC_b = b \cdot G + \rho \cdot H.$$

2.3 Balance Ciphertext Linkage

Let (B_1, B_2) denote the sender's existing on-ledger balance ciphertext under the sender's public key P_A. For some historical randomness $r_b \in \mathbb{Z}_q$:

$$B_1 = r_b \cdot G, \qquad B_2 = b \cdot G + r_b \cdot P_A.$$

Since (B_1, B_2) is already on-ledger, the prover may not know the original r_b. However, the sender knows $sk_A \in \mathbb{Z}_q$ with $P_A = sk_A \cdot G$, so the balance ciphertext satisfies:

$$B_2 - b \cdot G = sk_A \cdot B_1.$$

This allows the sigma proof to link PC_b to the balance ciphertext using sk_A rather than r_b.

2.4 Combined Relation

The prover's witness is

$$w = (m,\ r,\ b,\ \rho,\ sk_A).$$

The combined relation for ConfidentialMPTSend is

$$\mathcal{R}_{\text{send}} = \left\{ (\mathbf{x},\ w) \;\middle|\; \begin{aligned} C_1 &= r \cdot G, \\ C_{2,i} &= m \cdot G + r \cdot P_i \quad (i = 1, \ldots, n), \\ PC_m &= m \cdot G + r \cdot H, \\ PC_b &= b \cdot G + \rho \cdot H, \\ P_A &= sk_A \cdot G, \\ B_2 - b \cdot G &= sk_A \cdot B_1 \end{aligned} \right\}, \tag{1}$$

where $\mathbf{x} = (C_1, \{C_{2,i}\}_{i=1}^n, PC_m, PC_b, B_1, B_2, P_1, \ldots, P_n, P_A, H)$.

This relation captures three consistency requirements simultaneously: (i) all amount ciphertexts encrypt the same value m using shared randomness r; (ii) the Pedersen commitment PC_m commits to the same amount m; and (iii) the Pedersen commitment PC_b commits to the same balance b as the existing balance ciphertext.

The aggregated Bulletproof separately proves $m \in [0, 2^{64})$ and $b - m \in [0, 2^{64})$ using commitments PC_m and $PC_b - PC_m = (b - m) \cdot G + (\rho - r) \cdot H$.

3 AND-Composed Compact-Form Sigma Protocol

3.1 Interactive Protocol

Protocol 3.1 (AND-composed Sigma protocol Σ_{send})**.** On public input $\mathbf{x}$ and private input $w = (m, r, b, \rho, sk_A)$:

1. **Commit.** Prover samples nonces $\alpha_m, \alpha_r, \alpha_b, \alpha_\rho, \alpha_{sk} \xleftarrow{R} \mathbb{Z}_q$ and computes $n+5$ commitment elements:

$$T_1 = \alpha_r \cdot G, \tag{2}$$
$$T_{2,i} = \alpha_m \cdot G + \alpha_r \cdot P_i \quad (i = 1, \ldots, n), \tag{3}$$
$$T_m = \alpha_m \cdot G + \alpha_r \cdot H, \tag{4}$$
$$T_b = \alpha_b \cdot G + \alpha_\rho \cdot H, \tag{5}$$
$$T_{sk,1} = \alpha_{sk} \cdot G, \tag{6}$$
$$T_{sk,2} = \alpha_b \cdot G + \alpha_{sk} \cdot B_1. \tag{7}$$

   Prover sends $(T_1, T_{2,1}, \ldots, T_{2,n}, T_m, T_b, T_{sk,1}, T_{sk,2})$.

2. **Challenge.** Verifier sends $e \xleftarrow{R} \mathbb{Z}_q$.

3. **Response.** Prover computes and sends:

$$z_m = \alpha_m + e \cdot m, \tag{8}$$
$$z_r = \alpha_r + e \cdot r, \tag{9}$$
$$z_b = \alpha_b + e \cdot b, \tag{10}$$
$$z_\rho = \alpha_\rho + e \cdot \rho, \tag{11}$$
$$z_{sk} = \alpha_{sk} + e \cdot sk_A. \tag{12}$$

4. **Verification.** Verifier accepts iff all of the following hold:

$$z_r \cdot G = T_1 + e \cdot C_1, \tag{13}$$
$$z_m \cdot G + z_r \cdot P_i = T_{2,i} + e \cdot C_{2,i} \quad \forall\, i, \tag{14}$$
$$z_m \cdot G + z_r \cdot H = T_m + e \cdot PC_m, \tag{15}$$
$$z_b \cdot G + z_\rho \cdot H = T_b + e \cdot PC_b, \tag{16}$$
$$z_{sk} \cdot G = T_{sk,1} + e \cdot P_A, \tag{17}$$
$$z_b \cdot G + z_{sk} \cdot B_1 = T_{sk,2} + e \cdot B_2. \tag{18}$$

This is the standard AND-composition [CDS94] of the same-plaintext, amount-linkage, and balance-linkage protocols under a shared challenge. The interactive proof transmits $n+5$ group elements and 5 scalars.

Uncompressed proof size. The merged proof transmits $(n+5)$ group elements and 5 scalars:

$$(n+5) \times 33 + 5 \times 32 \;=\; \begin{cases} 424 \text{ bytes} & (n = 3), \\ 457 \text{ bytes} & (n = 4). \end{cases}$$

This compares to 586 bytes ($n = 3$) or 619 bytes ($n = 4$) for three separate proofs—a 28% reduction from witness sharing alone.

3.2 Compact (Challenge) Form

Each verification equation can be rearranged to express the commitment in terms of the challenge and responses:

$$T_1 = z_r \cdot G - e \cdot C_1, \tag{19}$$
$$T_{2,i} = z_m \cdot G + z_r \cdot P_i - e \cdot C_{2,i} \quad \forall\, i, \tag{20}$$
$$T_m = z_m \cdot G + z_r \cdot H - e \cdot PC_m, \tag{21}$$
$$T_b = z_b \cdot G + z_\rho \cdot H - e \cdot PC_b, \tag{22}$$
$$T_{sk,1} = z_{sk} \cdot G - e \cdot P_A, \tag{23}$$
$$T_{sk,2} = z_b \cdot G + z_{sk} \cdot B_1 - e \cdot B_2. \tag{24}$$

In the Fiat–Shamir transform, the challenge is:

$$e = \mathcal{H}\big(\texttt{"CMPT\_SEND\_SIGMA"} \parallel P_1 \| \cdots \| P_n \| P_A \parallel C_1 \| C_{2,1} \| \cdots \| C_{2,n} \parallel PC_m \| PC_b \| B_1 \| B_2 \parallel T_1 \| T_{2,1} \| \cdots \| T_{2,n} \| T_m \| T_b \| T_{sk,1} \| T_{sk,2} \parallel \mathsf{TransactionContextID}\big). \tag{25}$$

Definition 3.2 (Compact-form proof Π_{send})**.** The non-interactive proof in compact form is

$$\Pi_{\text{send}} = (e,\, z_m,\, z_r,\, z_b,\, z_\rho,\, z_{sk}) \;\in\; \mathbb{Z}_q^6.$$

Verification:

1. Reconstruct all $n + 5$ commitments via Equations (19) to (24).
2. Recompute $e' = \mathcal{H}(\cdots)$ using Equation (25) with the reconstructed commitments.
3. Accept iff $e' = e$.

Proof size. $6 \times 32 = \mathbf{192}$ bytes (one challenge, five responses; all $\mathbb{Z}_q$ scalars). The proof size is **independent of** n; verification cost remains $O(n)$ group operations.

Remark 3.3 (Equivalence of commitment and compact forms)**.** The two forms define equivalent verification procedures: a proof is valid in one form if and only if the corresponding proof is valid in the other. In the commitment form, the verifier computes e from the hash and checks the verification equations; in the compact form, the verifier reconstructs the commitments from the same equations and checks the hash. A valid commitment-form proof converts to compact form by dropping the commitments and prepending e, and vice versa.

4 Security

We prove security properties of the interactive protocol Σ_{send} (Theorem 3.1), then lift to the non-interactive setting via the Fiat–Shamir transform. Throughout, λ denotes the security parameter, $\mathbb{G}$ is the secp256k1 group of prime order q ($\approx 2^{256}$), with generators G and H such that $\log_G H$ is unknown.

4.1 Completeness

Theorem 4.1 (Completeness)**.** *If the prover holds a valid witness $w = (m, r, b, \rho, sk_A)$ for a statement $\mathbf{x} \in \mathcal{R}_{\text{send}}$, then the verifier always accepts.*

Proof. We verify each equation.

Equation (13): $z_r \cdot G = (\alpha_r + er) \cdot G = \alpha_r \cdot G + e \cdot r \cdot G = T_1 + e \cdot C_1$. ✓

Equation (14), for each i:

$$\begin{aligned} z_m \cdot G + z_r \cdot P_i &= (\alpha_m + em) \cdot G + (\alpha_r + er) \cdot P_i \\ &= (\alpha_m \cdot G + \alpha_r \cdot P_i) + e \cdot (m \cdot G + r \cdot P_i) \\ &= T_{2,i} + e \cdot C_{2,i}. \checkmark \end{aligned}$$

Equation (15): $z_m \cdot G + z_r \cdot H = (\alpha_m \cdot G + \alpha_r \cdot H) + e \cdot (m \cdot G + r \cdot H) = T_m + e \cdot PC_m$. ✓

Equation (16): $z_b \cdot G + z_\rho \cdot H = (\alpha_b \cdot G + \alpha_\rho \cdot H) + e \cdot (b \cdot G + \rho \cdot H) = T_b + e \cdot PC_b$. ✓

Equation (17): $z_{sk} \cdot G = (\alpha_{sk} + e \cdot sk_A) \cdot G = \alpha_{sk} \cdot G + e \cdot sk_A \cdot G = T_{sk,1} + e \cdot P_A$. ✓

Equation (18):

$$\begin{aligned} z_b \cdot G + z_{sk} \cdot B_1 &= (\alpha_b + eb) \cdot G + (\alpha_{sk} + e \cdot sk_A) \cdot B_1 \\ &= (\alpha_b \cdot G + \alpha_{sk} \cdot B_1) + e \cdot (b \cdot G + sk_A \cdot B_1) \\ &= T_{sk,2} + e \cdot B_2. \checkmark \end{aligned}$$

□

4.2 Special Soundness

Theorem 4.2 (2-special soundness)**.** *Given two accepting transcripts*

$$\begin{aligned} &(T_1, T_{2,i}, T_m, T_{sk,1}, T_b, T_{sk,2},\ e,\ z_r, z_m, z_{sk}, z_\rho, z_b) \quad \textit{and} \\ &(T_1, T_{2,i}, T_m, T_{sk,1}, T_b, T_{sk,2},\ e',\ z'_r, z'_m, z'_{sk}, z'_\rho, z'_b) \end{aligned}$$

with the same first-round commitments and $e \neq e'$, one can efficiently extract a witness (m, r, b, ρ, sk_A) satisfying all relations of Equation (1).

Proof. Define $\Delta e = e - e'$, which is nonzero and hence invertible in $\mathbb{Z}_q$. Set:

$$m = \frac{z_m - z'_m}{\Delta e}, \quad r = \frac{z_r - z'_r}{\Delta e}, \quad b = \frac{z_b - z'_b}{\Delta e}, \quad \rho = \frac{z_\rho - z'_\rho}{\Delta e}, \quad sk_A = \frac{z_{sk} - z'_{sk}}{\Delta e}.$$

Relation 1 ($C_1 = r \cdot G$): Subtracting the two instances of Equation (13): $(z_r - z'_r) \cdot G = (e - e') \cdot C_1$, so $r \cdot G = C_1$. ✓

Relation 2 ($C_{2,i} = m \cdot G + r \cdot P_i$ for all i): Subtracting the two instances of Equation (14): $(z_m - z'_m) \cdot G + (z_r - z'_r) \cdot P_i = (e - e') \cdot C_{2,i}$, so $m \cdot G + r \cdot P_i = C_{2,i}$. ✓

Relation 3 ($PC_m = m \cdot G + r \cdot H$): From Equation (15): $(z_m - z'_m) \cdot G + (z_r - z'_r) \cdot H = (e - e') \cdot PC_m$, so $m \cdot G + r \cdot H = PC_m$. ✓

Relation 4 ($PC_b = b \cdot G + \rho \cdot H$): From Equation (16): $(z_b - z'_b) \cdot G + (z_\rho - z'_\rho) \cdot H = (e - e') \cdot PC_b$, so $b \cdot G + \rho \cdot H = PC_b$. ✓

Relation 5 ($P_A = sk_A \cdot G$): From Equation (17): $(z_{sk} - z'_{sk}) \cdot G = (e - e') \cdot P_A$, so $sk_A \cdot G = P_A$. ✓

Relation 6 ($B_2 - b \cdot G = sk_A \cdot B_1$): From Equation (18): $(z_b - z'_b) \cdot G + (z_{sk} - z'_{sk}) \cdot B_1 = (e - e') \cdot B_2$, so $b \cdot G + sk_A \cdot B_1 = B_2$. ✓

All six relations are satisfied by the extracted witness. □

4.3 Honest-Verifier Zero-Knowledge

Theorem 4.3 (HVZK)**.** Σ_{send} *is special honest-verifier zero-knowledge for* $\mathcal{R}_{\text{send}}$.

Proof. We construct a simulator $\mathcal{S}$ that, given any challenge $e \in \mathbb{Z}_q$ and a valid statement (but no witness), outputs a transcript indistinguishable from a real one.

Simulator $\mathcal{S}(e)$:

1. Sample $z_m, z_r, z_b, z_\rho, z_{sk} \xleftarrow{R} \mathbb{Z}_q$.
2. Set commitments via the reconstruction equations Equations (19) to (24).
3. Output the full transcript $(T_1, T_{2,i}, T_m, T_{sk,1}, T_b, T_{sk,2},\ e,\ z_r, z_m, z_{sk}, z_\rho, z_b)$.

Verification. By construction, the transcript satisfies all verification equations Equations (13) to (18).

Distribution. In a real transcript, the nonces $\alpha_m, \alpha_r, \alpha_b, \alpha_\rho, \alpha_{sk}$ are uniform in $\mathbb{Z}_q$, and the responses are affine functions of the nonces. For any fixed e and witness, the map $(\alpha_m, \alpha_r, \alpha_b, \alpha_\rho, \alpha_{sk}) \mapsto (z_m, z_r, z_b, z_\rho, z_{sk})$ is a bijection on $\mathbb{Z}_q^5$. Therefore the responses are uniformly distributed, and the commitments are deterministic functions of the statement, challenge, and responses. The simulated transcript has the same structure. The two distributions are identical. □

4.4 Non-Interactive Zero-Knowledge in the Random Oracle Model

Theorem 4.4 (NIZK in the ROM)**.** *The Fiat–Shamir transform of* Σ_{send} *(in either commitment or compact form) is a non-interactive zero-knowledge proof of knowledge for* $\mathcal{R}_{\text{send}}$ *in the random oracle model, under the discrete logarithm assumption in* $\mathbb{G}$.

Proof sketch. **Knowledge soundness (via the forking lemma).** Let $\mathcal{A}$ be a PPT adversary that produces an accepting proof with non-negligible probability ε. Apply the general forking lemma [BN06]: run $\mathcal{A}$ once to obtain an accepting transcript, then fork with a fresh random oracle response at the challenge query. With probability $\geq \varepsilon^2/q_H - \varepsilon/q$, both runs produce accepting proofs with the same commitments but distinct challenges. By Theorem 4.2, the extractor recovers (m, r, b, ρ, sk_A).

Zero-knowledge (via RO simulation). The simulator samples uniform responses, reconstructs commitments via Equations (19) to (24) using a random e, and programs the random oracle to return e on the appropriate input. Programming succeeds except with probability $q_H/q = \mathsf{negl}(\lambda)$. □

Remark 4.5 (Long-lived secret sk_A)**.** The compact form leaks no more about the long-lived key sk_A than the commitment form: both admit the same witness extraction (Theorem 4.2), and the HVZK simulator (Theorem 4.3) produces transcripts without any witness. In the non-interactive setting, Fiat–Shamir prevents an adversary from obtaining two transcripts with the same commitments but distinct challenges (requiring a hash collision). The security of sk_A reduces to the discrete logarithm assumption, identically to a standalone Schnorr PoK.

5 Constant-Size Shared-Randomness Equality Proof

The combined sigma protocol of Section 3 proves ciphertext equality as part of a larger conjunction that includes the balance linkage. In some contexts—standalone equality attestations, or multi-auditor scenarios where n is large—it is useful to have a proof of shared-randomness equality alone, with proof size *independent of* n. The following construction, due to Cenk [Cen26], achieves this.

5.1 Algebraic Cancellation

Let $P_1, \ldots, P_n \in \mathbb{G}$ be distinct public keys and let $r \xleftarrow{R} \mathbb{Z}_q$. The sender encrypts a message $M = m \cdot G$ under shared randomness, producing n ciphertexts $E_i = (C, D_i)$ with

$$C = r \cdot G, \qquad D_i = M + r \cdot P_i, \quad i = 1, \ldots, n.$$

Since C is shared, all ciphertexts have a common first component. The goal is to prove that all D_i encrypt the same M without revealing M or r.

Define differences relative to the first ciphertext:

$$\Delta P_i = P_i - P_1, \qquad \Delta D_i = D_i - D_1, \qquad i = 2, \ldots, n.$$

Then $\Delta D_i = r \cdot \Delta P_i$ (the message cancels). The problem reduces to a multi-base DLEQ: prove knowledge of r such that $C = r \cdot G$ and $\Delta D_i = r \cdot \Delta P_i$ for all $i \geq 2$.

5.2 The (c, s) Variant

Because all DLEQ relations share a *single* scalar witness r, a single Schnorr response suffices.

Protocol 5.1 (π_{eq}: constant-size shared-r equality)**.** On public input $(C,\ \Delta D_2, \ldots, \Delta D_n,\ \Delta P_2, \ldots, \Delta P_n)$ and private input r:

1. **Commit.** Sample $k \xleftarrow{R} \mathbb{Z}_q$. Compute $R_G = k \cdot G$ and $R_i = k \cdot \Delta P_i$ for $i = 2, \ldots, n$.
2. **Challenge (Fiat–Shamir).**

$$c = H\big(C \| \Delta D_2 \| \cdots \| \Delta D_n \| \Delta P_2 \| \cdots \| \Delta P_n \| R_G \| R_2 \| \cdots \| R_n\big).$$

The public bases ΔP_i are included so that the challenge binds to the full public statement, not just to the commitments ΔD_i and announcements R_i. Omitting them would let a malicious prover choose $\Delta P_i = s^{-1}(R_i + c\,\Delta D_i)$ for arbitrary R_i after seeing c, making the verifier's reconstruction equation hold for a statement different from the one the prover committed to.

3. **Response.** $s = k + c \cdot r$.
4. **Proof.** $\pi_c = (c, s)$.

Verification. The verifier reconstructs:

$$R'_G = s \cdot G - c \cdot C, \qquad R'_i = s \cdot \Delta P_i - c \cdot \Delta D_i \quad (i = 2, \ldots, n),$$

and checks $c \stackrel{?}{=} H\big(C \| \Delta D_2 \| \cdots \| \Delta D_n \| \Delta P_2 \| \cdots \| \Delta P_n \| R'_G \| R'_2 \| \cdots \| R'_n\big)$.

Proof size. $\pi_c = (c, s) \in \mathbb{Z}_q^2$: **64 bytes**, independent of n. Verification cost is $\mathcal{O}(n)$ scalar multiplications, but the proof transmitted on-chain is $\mathcal{O}(1)$.

Security. Completeness and special soundness follow from the standard Schnorr argument applied to the multi-base DLEQ. The single response s suffices because all n relations share the same witness r; the extractor recovers $r = (s - s')/(c - c')$ from two accepting transcripts.

5.3 Relationship to the Combined Sigma Proof

For the full ConfidentialMPTSend proof, the combined sigma protocol of Section 3 already handles shared-randomness equality as part of the larger conjunction. The compact form gives 192 bytes for the full proof (equality + amount linkage + balance linkage), independent of n.

The π_{eq} construction is therefore not needed for ConfidentialMPTSend at small fixed n. It becomes relevant in two scenarios:

1. **Large** n (e.g., multi-auditor extensions where $n \gg 4$): while the compact sigma proof's byte count is already n-independent, its verification cost grows as $\mathcal{O}(n)$. Separating the shared-r equality proof (64 bytes) from the remaining balance relations could enable more modular verification strategies.
2. **Standalone equality attestations** outside of ConfidentialMPTSend (e.g., proving re-encryption correctness during key rotation).

6 Implementation Considerations

6.1 Deterministic Nonces

The prover's nonces $(\alpha_m, \alpha_r, \alpha_b, \alpha_\rho, \alpha_{sk})$ must be sampled uniformly and independently for each proof instance. Reusing a nonce across two proofs with distinct challenges—or across two distinct statements with the same challenge—allows trivial witness extraction via the special soundness equations.

Because the compact form transmits $(e, z_m, z_r, z_b, z_\rho, z_{sk})$ rather than the commitments, a nonce-reuse attack is not detectable from the proof alone; the resulting proofs would simply be valid proofs of distinct statements that happen to share the underlying nonces. An adversary observing both proofs can extract the witnesses.

Remark 6.1 (Synthetic nonces)**.** We recommend *deterministic nonce generation* following RFC 6979 [Por13]:

$$(\alpha_m, \ldots, \alpha_{sk}) \leftarrow \mathsf{HKDF}(sk_A \| r \| m \| b \| \rho \| \text{statement} \| \text{fresh entropy}).$$

Including the full witness ensures nonces are unique per transaction. The optional fresh entropy provides defense-in-depth.

6.2 Fiat–Shamir Domain Separation

The hash input for the challenge (Equation (25)) must include:

1. A fixed domain-separation tag (`"CMPT_SEND_SIGMA"`).
2. *All* public-key and ciphertext components: the recipient keys $P_1, \ldots, P_n$, the sender key P_A, the ciphertext bundle $C_1, C_{2,1}, \ldots, C_{2,n}$, the commitments PC_m, PC_b, and the balance ciphertext (B_1, B_2).
3. All first-round commitments $T_1, T_{2,1}, \ldots, T_{2,n}, T_m, T_b, T_{sk,1}, T_{sk,2}$.
4. The TransactionContextID for replay protection.

Omitting any statement element from the hash would allow cross-protocol attacks. The domain-separation tag prevents collisions with other Fiat–Shamir proofs in the CMPT protocol.

6.3 Verification Cost

Compact-form and commitment-form verification require the same number of multi-scalar multiplications. The verifier performs identical linear combinations, just in *reconstruction* order rather than *check* order. The only additional cost is a hash evaluation to compute e', negligible compared to the group operations.

For general n, verification reconstructs $n+5$ commitments via $n+3$ double-exponentiations, 1 triple-exponentiation (for $T_m = z_m G + z_r H - e \cdot PC_m$), 1 single-exponentiation (for $T_{sk,1}$), and 1 hash evaluation. At $n = 4$: 11 multi-scalar multiplications total; at $n = 3$: 10.

Remark 6.2 (Batch verification tradeoff)**.** Compact-form proofs do not support batch verification in the standard way (random linear combination of independent verification equations across transactions), because the commitment points are *reconstructed* rather than transmitted independently. In commitment form, the verifier can batch the linear check $T + e \cdot S \stackrel{?}{=} z \cdot B$ across proofs by drawing random weights; in compact form, each proof's commitments depend on its own challenge and responses, so no cross-proof linear structure is available. In practice this tradeoff is favourable: the on-chain proof size reduction (from eliminating group elements) outweighs the marginal throughput loss, since network latency and transaction-size limits dominate validator cost in the XRPL ledger-close model.

7 Transaction Size Analysis

All sizes use compressed secp256k1 points (33 bytes) and $\mathbb{Z}_q$ scalars (32 bytes). We give results for both $n = 4$ recipients (sender, receiver, issuer, auditor—the production configuration) and $n = 3$ (no auditor). The only n-dependent component is the ciphertext data: each additional recipient adds one $C_{2,i}$ group element (+33 bytes). The compact sigma proof is n-independent.

Table 1: Sigma-proof size for ConfidentialMPTSend (standard EC-ElGamal). Each row shows sigma overhead, excluding ciphertext data and Bulletproof. The merged and separate forms grow by one group element (+33 B) per additional recipient; the compact form is n-independent.

	Separate (commit. form)	Merged (commit. form)	Compact form
$n = 4$ *(production: sender, receiver, issuer, auditor)*			
Group elements	$11 \times 33 = 363$	$9 \times 33 = 297$	0
Scalars	$8 \times 32 = 256$	$5 \times 32 = 160$	$6 \times 32 = 192$
Total	**619**	**457**	**192**
$n = 3$ *(no auditor)*			
Group elements	$10 \times 33 = 330$	$8 \times 33 = 264$	0
Scalars	$8 \times 32 = 256$	$5 \times 32 = 160$	$6 \times 32 = 192$
Total	**586**	**424**	**192**

7.1 Sigma-Proof Comparison

Table 1 compares the sigma-proof overhead across the three proof formats.
For $n = 4$, merging alone saves $619 - 457 = 162$ bytes by eliminating duplicate commitments and responses for shared witnesses. Compact form saves a further $457 - 192 = 265$ bytes by eliminating all group-element commitments. The total reduction from separate proofs to compact form is $619 - 192 = 427$ bytes (69%). The compact-form proof size is 192 bytes regardless of n, since only scalar responses are transmitted.

7.2 Full Transaction Comparison

Table 2: Full ConfidentialMPTSend transaction size breakdown, standard EC-ElGamal with compact sigma.

Component	$n = 4$	$n = 3$	Notes
Shared C_1	33	33	1×33
Recipient ciphertexts $C_{2,i}$	132	99	$n \times 33$
Amount commitment PC_m	33	33	1×33
Balance commitment PC_b	33	33	1×33
Compact sigma proof Π_{send}	192	192	6×32
Aggregated Bulletproof	754	754	unchanged
Total	**1177**	**1144**	$\Delta = 33$

7.3 Progressive Reductions

Table 3 shows the full optimization path from the CMPT specification baseline. The compact sigma protocol is the primary optimisation adopted in this document; the Bulletproofs++ upgrade [EKRN23] is an orthogonal enhancement analysed in [Dor26b].

Table 3: Progressive ConfidentialMPTSend transaction size reductions. The $n = 4$ column is the production configuration; $n = 3$ (no auditor) is shown for comparison. Each step from $n = 3$ to $n = 4$ adds exactly 33 bytes (one $C_{2,i}$ group element).

Configuration	n=4	n=3	vs. (A)	
(A) Per-CT layout, separate proofs	1604	1604	—	
(B) Shared C_1, separate proofs	1604	1538	0/−66	
(B′) (B) + merged Σ (commit. form)	1442	1376	−162	(10%)
(B″) (B) + compact Σ	**1177**	**1144**	**−427/ − 460**	**(27/29%)**
(C) (B″) + Bulletproofs++	∼ 905	∼ 872	∼−699/ − 732	(44/46%)

Note that the $n = 4$ baseline (A) coincidentally also equals 1604 bytes: Cenk's shared-C_1 formulation with 4 recipients happens to match Dore's per-CT formulation with 3 recipients (the extra $C_{2,4}$ adds 33 B while deduplicating C_1 saves 33 B, and the separate same-plaintext proof adds 33 B while losing one redundant $C_{1,i}$ saves 33 B).

The compact sigma optimisation alone recovers 427–460 bytes—more than 60% of the total potential saving including BP++. It requires no change to the encryption scheme, no new cryptographic primitives, and no specification of new ciphertext formats; only the proof serialization changes.

8 Extension to Other Transaction Types

The preceding analysis targets ConfidentialMPTSend, which carries the heaviest proof bundle. The remaining confidential transaction types—Convert, ConvertBack, and Clawback—operate at the public/confidential boundary: the transfer amount m is publicly known, eliminating the need for same-plaintext equality proofs and, in most cases, range proofs. We show that compact-form techniques apply to these lighter transactions as well.

8.1 Convert

In ConfidentialMPTConvert, a holder moves a publicly known amount m from cleartext into a confidential balance. Because m appears in plaintext on-chain and the holder generates the encryption randomness r themselves, the production CMPT specification uses *deterministic verification* rather than a zero-knowledge proof: the transaction includes a mandatory 32-byte `BlindingFactor` field carrying r, and validators check

$$C_1 \stackrel{?}{=} r \cdot G \qquad \text{and} \qquad C_{2,i} \stackrel{?}{=} m \cdot G + r \cdot pk_i$$

for each recipient key pk_i. No sigma proof is required or defined for Convert in the production protocol.

For reference, the natural sigma proof for this relation—a single Schnorr proof of knowledge of r (Π_{plain})—is serialised in commitment form as $(T_1, T_2, s) \in \mathbb{G}^2 \times \mathbb{Z}_q$ costing $2 \times 33 + 32 = 98$ bytes, or in compact form as

$$\Pi_{\text{plain}} = (e,\, z_r) \;\in\; \mathbb{Z}_q^2, \qquad \text{costing } 2 \times 32 = \mathbf{64} \text{ bytes},$$

with reconstruction:

$$T_1 = z_r \cdot G - e \cdot C_1, \tag{26}$$
$$T_2 = z_r \cdot P_A - e \cdot (C_2 - m \cdot G). \tag{27}$$

This proof would apply in contexts where the prover cannot or should not disclose r on-chain; it is not part of the production CMPT transaction protocol. Clawback does not admit the r-disclosure shortcut, since the issuer proves knowledge of sk_{iss}, not randomness (see Section 8.2).

8.2 Clawback

In ConfidentialMPTClawback, the issuer reclaims a publicly known amount m from a holder's confidential balance. Unlike Convert, the issuer proves knowledge of the issuer secret key sk_{iss} rather than randomness. Concretely, the issuer proves:

$$P_{\text{iss}} = sk_{\text{iss}} \cdot G \quad \text{and} \quad C_2 - m \cdot G = sk_{\text{iss}} \cdot C_1,$$

i.e. a Chaum–Pedersen equality-of-discrete-logs proof. In the current commitment form this is serialised as $(T_1, T_2, s) \in \mathbb{G}^2 \times \mathbb{Z}_q$ costing 98 bytes.

In compact form, the proof becomes

$$\Pi_{\text{claw}} = (e,\, z_{sk}) \;\in\; \mathbb{Z}_q^2, \qquad \text{costing } 2 \times 32 = \mathbf{64} \text{ bytes},$$

with reconstruction:

$$T_1 = z_{sk} \cdot G - e \cdot P_{\text{iss}}, \tag{28}$$
$$T_2 = z_{sk} \cdot C_1 - e \cdot (C_2 - m \cdot G). \tag{29}$$

The Fiat–Shamir hash binds the domain tag `CMPT_CLAWBACK_SIGMA`, the statement points P_{iss}, C_1, C_2, $m \cdot G$ (serialised as a 33-byte compressed point), the commitments T_1, T_2, and the transaction context TransactionContextID.

8.3 ConvertBack

In ConfidentialMPTConvertBack, a holder withdraws a publicly known amount m from a confidential balance back to cleartext. Because m is public, the withdrawal ciphertext randomness r_w is disclosed via the `BlindingFactor` field and verified deterministically by the validator (checking $C_{1,w} = r_w \cdot G$ and $C_{2,w} = m \cdot G + r_w \cdot P_A$), eliminating the need for a zero-knowledge proof of ciphertext correctness. The sigma proof therefore covers only the balance linkage relation. A single-value Bulletproof (688 bytes) then proves $b - m \in [0, 2^{64})$.

Balance linkage relation

The zero-knowledge proof establishes that the Pedersen commitment PC_b encodes the same balance b as the sender's on-ledger balance ciphertext (B_1, B_2):

$$\mathcal{R}_{\text{bal}} = \left\{ (\mathbf{x},\; w) \;\middle|\; \begin{array}{r} P_A = sk_A \cdot G, \\ B_2 - b \cdot G = sk_A \cdot B_1, \\ PC_b = b \cdot G + \rho \cdot H \end{array} \right\}, \tag{30}$$

where the public statement is $\mathbf{x} = (P_A, B_1, B_2, PC_b, H)$ and the witness is $w = (b,\ sk_A,\ \rho) \in \mathbb{Z}_q^3$. This is precisely the balance sub-relation of $\mathcal{R}_{\text{send}}$, now standing alone since the amount witness (m, r) is not hidden.

AND-composed compact-form protocol

The prover samples nonces $(t_b,\ t_{sk},\ t_\rho) \xleftarrow{\$} \mathbb{Z}_q^3$ and computes three commitments:

$$\begin{aligned} T_{sk,1} &= t_{sk} \cdot G, \\ T_{sk,2} &= t_b \cdot G + t_{sk} \cdot B_1, \\ T_b &= t_b \cdot G + t_\rho \cdot H. \end{aligned}$$

Challenge and responses follow the same pattern as Section 3: a single Fiat–Shamir challenge e binds all three commitments, and responses are $z_b = t_b + e \cdot b$, $z_{sk} = t_{sk} + e \cdot sk_A$, $z_\rho = t_\rho + e \cdot \rho$.

The Fiat–Shamir hash is:

$$e = \mathcal{H}\big(\texttt{"CMPT\_CONVERTBACK\_SIGMA"} \parallel P_A \parallel B_1 \| B_2 \parallel PC_b \parallel T_{sk,1} \| T_{sk,2} \| T_b \parallel \mathsf{TransactionContextID}\big). \quad (31)$$

Definition 8.1 (Compact-form proof Π_{cb})**.** The non-interactive proof in compact form is

$$\Pi_{\text{cb}} = (e,\ z_b,\ z_\rho,\ z_{sk}) \ \in \ \mathbb{Z}_q^4.$$

Verification:

1. Reconstruct commitments:

$$T_{sk,1} = z_{sk} \cdot G - e \cdot P_A, \quad (32)$$
$$T_{sk,2} = z_b \cdot G + z_{sk} \cdot B_1 - e \cdot B_2, \quad (33)$$
$$T_b = z_b \cdot G + z_\rho \cdot H - e \cdot PC_b. \quad (34)$$

2. Recompute e' from the Fiat–Shamir hash (Equation (31)) with the reconstructed commitments.

3. Accept iff $e' = e$.

Proof size. $4 \times 32 = \mathbf{128}$ bytes (one challenge, three responses). The current commitment-form cost for balance linkage alone is 195 bytes, yielding a saving of 67 bytes.

Remark 8.2 (Security)**.** The security argument for Π_{cb} is analogous to Section 4: the witnesses (b, sk_A, ρ) are independent, so 2-special soundness extracts each from two accepting transcripts with distinct challenges. HVZK follows from the uniform distribution of the nonces.

8.4 Cross-Transaction Summary

Table 4 summarises the compact-form savings across all four confidential transaction types. The absolute savings are largest for Send (427 bytes), as expected given its richer proof structure. For ConvertBack, the 67-byte reduction yields a total proof payload of $128 + 688 = 816$ bytes, down from $195 + 688 = 883$ bytes (8%). Convert and Clawback see modest absolute savings (34 bytes each), though the percentage reduction in sigma overhead is comparable.

The compact-form technique is uniform: the same design pattern—AND-compose all sigma relations, transmit $(e, \mathbf{z})$ instead of $(\mathbf{T}, \mathbf{z})$, reconstruct commitments during verification—applies to every transaction type without exception.

Table 4: Sigma-proof sizes across transaction types. Range proofs (Bulletproof) are unchanged and shown separately. In production, Convert uses r-disclosure (Section 8.1) rather than a sigma proof; the Convert row shows hypothetical compact-form sizes for reference.

Transaction type	Sigma (current)	Sigma (compact)	Range proof	Sigma saving
Send (n=4)	619	192	754	427 (69%)
ConvertBack	195	128	688	67 (34%)
Convert	98	64	—	34 (35%)
Clawback	98	64	—	34 (35%)

8.5 Implementation Changes

Adopting compact sigma for the non-Send transaction types requires coordinated changes in the cryptographic library (`mpt-crypto`) and the ledger node (`rippled`).

Changes in `mpt-crypto`

The `mpt-crypto` library already implements each building-block sigma protocol as a separate prove/verify pair: `secp256k1_equality_plaintext_{prove,verify}` for the plaintext PoK (Π_{plain}, 98 bytes) and `secp256k1_elgamal_pedersen_link_{prove,verify}` for the linkage proof (Π_{link}, 195 bytes). The different transaction types compose these building blocks: Convert uses deterministic r-disclosure (no sigma proof; see Section 8.1); Clawback uses a Chaum–Pedersen equality-of-discrete-logs proof for sk_{iss}; ConvertBack uses $\Pi_{\text{link-B}}$ plus a Bulletproof; Send uses all three plus the shared-r equality proof.

The compact sigma implementation for Send (PR #18) establishes the pattern: a single new source file (`proof_compact_standard.c`) with dedicated prove/verify entry points, a size constant, and a Fiat–Shamir domain tag distinct from the commitment-form proofs. No changes to the underlying scalar or group-arithmetic layer are needed.

Convert requires no new source file: the production protocol uses deterministic r-disclosure via `BlindingFactor`, so no sigma prove/verify pair is defined. Following this convention, we propose one new source file per remaining transaction type:

Clawback (compact Π_{claw}).
Implemented in `proof_compact_clawback.c`. The proof shrinks from 98 to 64 bytes. The prover samples one nonce t_{sk}, computes $z_{sk} = t_{sk} + e \cdot sk_{\text{iss}}$, and serialises (e, z_{sk}). The verifier reconstructs T_1, T_2 via Equations (28) to (29) and re-hashes under the domain tag `CMPT_CLAWBACK_SIGMA`. Note that $m \cdot G$ enters the hash as a 33-byte compressed point, not a raw scalar.

ConvertBack (compact Π_{cb}).
A new source file (e.g. `proof_compact_convertback.c`) providing prove/verify entry points. Proof size: 128 bytes. Because the withdrawal ciphertext randomness r_w is disclosed via the `BlindingFactor` field and verified deterministically, the sigma proof covers only the balance linkage relation (Section 8.3). The implementation samples three nonces, computes three commitments, derives e under the domain tag `CMPT_CONVERTBACK_SIGMA`, computes three responses, and serialises (e, z_b, z_ρ, z_{sk}).

The Bulletproof range proof for $b - m \in [0, 2^{64})$ is unchanged; the existing Bulletproof prove/verify functions continue to operate on the Pedersen commitment $PC_b - m \cdot G$.

PR #18 also introduces `mpt_internal.h` with shared helpers (`generate_random_scalar`, `mpt_uint64_to_scalar`, etc.) that the new source files can reuse directly. In all cases, the old commitment-form functions on main are retained; the compact variants are strictly additive. The nonce-generation guidance of Section 6.1 (deterministic synthetic nonces incorporating the full witness) applies equally to ConvertBack and Clawback.

Changes in `rippled`

The ledger node calls `mpt-crypto` during transaction validation. Required changes:

1. **Proof-size constants.** The serialisation layer must accept the new proof sizes: 64 bytes for Convert and Clawback sigma proofs and 128 bytes for ConvertBack sigma proofs (replacing 98 and 195 respectively). These constants gate deserialisation and buffer allocation.

2. **Verification call sites.** Each transaction handler currently calls the commitment-form verify function. For ConfidentialMPTConvertBack, the two separate verify calls (plaintext proof and linkage proof) are replaced by a single call to the compact ConvertBack verifier. For Convert, a single call replaces the existing one with the only visible change being the proof buffer size. For Clawback, the call similarly replaces the existing one, but the underlying proof relation changes from knowledge of randomness to knowledge of sk_{iss}.

3. **TransactionContextID construction.** The Fiat–Shamir hash in each compact proof binds the TransactionContextID. The `rippled` code that constructs these context IDs is transaction-type-specific and is already in place; no change is needed unless the domain tag is incorporated into the context ID itself rather than the proof hash (the current design keeps them separate).

4. **Amendment gating.** Because the proof format is a consensus-visible change (validators must agree on the serialisation), the new compact formats must be gated behind an XRPL amendment. The amendment can cover all three transaction types simultaneously, or be staged (Send first, others in a follow-up), depending on rollout preferences. During the transition, validators must accept *both* commitment-form and compact-form proofs until the amendment activates, at which point only compact form is accepted.

Remark 8.3 (Scope relative to Send)**.** The compact sigma for Send is the higher-priority implementation target, since it delivers the largest absolute saving (427 bytes) on the most frequent transaction type. The Convert/ConvertBack/Clawback extensions described here are incremental: they reuse the same architectural pattern and can share infrastructure (e.g., the scalar serialisation helpers in `mpt_internal.h`) already introduced for Send. They can be implemented in a follow-up PR once the Send compact sigma has been validated.

A Twisted ElGamal Variant

We briefly present the compact sigma protocol for the twisted ElGamal construction of [Dor26a] and analyse why its advantage over standard EC-ElGamal shrinks under compact form. The

team has elected **not** to adopt twisted ElGamal for the CMPT specification, for the reasons summarised in Section A.5.

A.1 Combined Language

In twisted ElGamal, each ciphertext component $Y = r \cdot G + m \cdot H$ simultaneously serves as a Pedersen commitment, eliminating separate PC_m and PC_b. The refreshed remainder ciphertext (Y^*, X^*) uses a DDH proof to link to the balance. The combined relation is:

$$L_{\text{comb}}^{(n)} = \{(pk_i, Y, X_i, pk_A, D_X, D_Y) \mid \exists\, (r, m, sk_A) \in \mathbb{Z}_q^3 : \\ Y = r \cdot G + m \cdot H \ \wedge\ X_i = r \cdot pk_i\ \forall i \ \wedge\ pk_A = sk_A \cdot G \ \wedge\ D_X = sk_A \cdot D_Y\},$$

where D_X, D_Y are the difference ciphertext components. The witness is (r, m, sk_A)—only three scalars, compared to five for standard EC-ElGamal.

A.2 Compact-Form Protocol

The AND-composed sigma protocol has nonces (a, b, k) for (r, m, sk_A), with $n + 3$ first-round commitments:

$$T_Y = a \cdot G + b \cdot H, \qquad T_{X,i} = a \cdot pk_i \quad (i = 1, \ldots, n),$$
$$K_1 = k \cdot G, \qquad K_2 = k \cdot D_Y.$$

Responses: $z_1 = a + e \cdot r$, $z_2 = b + e \cdot m$, $z_3 = k + e \cdot sk_A$.

Compact form. $\Pi_{\text{comb}} = (e, z_1, z_2, z_3) \in \mathbb{Z}_q^4$: $4 \times 32 = \mathbf{128}$ bytes.

Verification. Reconstruct commitments:

$$T_Y = z_1 \cdot G + z_2 \cdot H - e \cdot Y, \qquad T_{X,i} = z_1 \cdot pk_i - e \cdot X_i,$$
$$K_1 = z_3 \cdot G - e \cdot pk_A, \qquad K_2 = z_3 \cdot D_Y - e \cdot D_X.$$

Check $e \stackrel{?}{=} \mathcal{H}(\cdots)$.

Security follows from the same arguments as Section 4, with the witness and nonce spaces having dimension 3 instead of 5.

A.3 The 64-Byte Gap

Table 5 compares the sigma overhead of both schemes across proof formats.
Two observations:

1. **Compact form benefits standard EC-ElGamal more.** Standard EC-ElGamal has more group-element commitments to eliminate (8 vs. 6 in merged form), yielding a larger absolute reduction (−232 B vs. −166 B).

2. **The twisted ElGamal advantage shrinks from 292 to 64 bytes.** In commitment form, twisted ElGamal saves bytes primarily by eliminating linkage proofs whose cost comes from group-element commitments (33 B each). Compact form removes *all* group-element commitments from *both* schemes. What remains is the irreducible per-witness

Table 5: Sigma-proof size comparison: standard vs. twisted EC-ElGamal. Commitment-form sizes depend on n; compact-form sizes do not.

	Separate (commit. form)		Merged (commit. form)		Compact form
	n=4	n=3	n=4	n=3	
Std. EC-ElGamal	619	586	457	424	192
Twisted ElGamal	327	294	327	294	128
Std. → Twisted savings	292	292	130	130	**64**

cost: each independent scalar witness contributes one 32-byte response. Twisted ElGamal has three witnesses (r, m, sk_A) whereas standard EC-ElGamal has five (m, r, b, ρ, sk_A), because the dual-purpose ciphertext/commitment structure of twisted ElGamal makes separate Pedersen commitments—and their associated witnesses ρ and b—unnecessary.

The 64-byte gap is **structural and irreducible**: it comes from the number of independent scalar witnesses, not from group elements. No further compression technique—including the $\mathcal{O}(1)$ equality proof of Section 5—can close it. Adding or removing auditors changes the ciphertext count by ± 33 bytes per recipient, identically for both schemes, so the relative comparison is unaffected by n.

A.4 Full Transaction Comparison

Table 6 shows the complete transaction sizes.

Table 6: Full ConfidentialMPTSend comparison under compact sigma. The Δ column (twisted vs. standard) is n-independent.

Component	Std. EG compact		Twisted EG compact		Δ
	n=4	n=3	n=4	n=3	
Ciphertext / commitments	231	198	231	198	0
Compact sigma proof	192	192	128	128	−64
Aggregated Bulletproof	754	754	754	754	0
Total	**1177**	**1144**	**1113**	**1080**	**−64**
With BP++ upgrade	∼ 905	∼ 872	∼ 841	∼ 816	∼ −64

A.5 Assessment

The 64-byte advantage of twisted ElGamal must be weighed against its costs:

(i) **New API surface.** Twisted ElGamal requires a generator-swap API (*value_gen* $= H$, *blinding_gen* $= G$) for both Pedersen commitments and Bulletproof range proofs. Passing generators in the wrong order silently produces insecure proofs.

(ii) **Audit scope.** The existing `mpt-crypto` library implements standard EC-ElGamal with substantial test coverage and is currently under audit. Twisted ElGamal requires auditing new encryption, decryption, and proof primitives.

(iii) **Specification changes.** The CMPT spec and XLS-96 are written for standard EC-ElGamal. Switching requires updating both documents, re-specifying all Fiat–Shamir transcripts, and revising the on-chain ciphertext format.

For context, the BP++ upgrade saves $\sim$ 272 bytes—more than 4$\times$ the twisted ElGamal advantage. At $n = 4$, standard EC-ElGamal with compact sigma and BP++ gives $\sim$ 905 bytes, compared to $\sim$ 841 bytes for twisted ElGamal with the same optimizations: a difference of 64 bytes (7%) on a total already below 1 KB.

Remark A.1 (Recommendation)**.** Given that the compact sigma optimization is adopted, the 64-byte saving from twisted ElGamal does not justify the specification, implementation, and audit costs. The team's decision to proceed with standard EC-ElGamal and compact sigma is sound.

References

[BN06] Mihir Bellare and Gregory Neven. Multi-signatures in the plain public-key model and a general forking lemma. In *ACM CCS 2006*, pages 390–399, 2006.

[BBB+18] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *IEEE Symposium on Security and Privacy*, pages 315–334, 2018.

[CDS94] Ronald Cramer, Ivan Damgård, and Berry Schoenmakers. Proofs of partial knowledge and simplified design of witness hiding protocols. In *CRYPTO '94*, volume 839 of *LNCS*, pages 174–187. Springer, 1994.

[CMA26] Murat Cenk, Aanchal Malhotra, and Ayo Akinyele. Confidential multi-purpose tokens on the XRP ledger. Ripple Research, February 2026. Internal specification, version 2026-02-01.

[Cen26] Murat Cenk. Proof unification and transcript compression for efficient confidential transfers. Ripple Research, March 2026. Internal working document.

[Dor26a] Tess Dore. Twisted ElGamal and shared randomness for confidential multi-purpose tokens on XRPL. RippleX Research, March 2026.

[Dor26b] Tess Dore. Bulletproofs++ upgrade for confidential multi-purpose tokens on XRPL. RippleX Research, March 2026.

[EKRN23] Liam Eagen, Sanket Kanjalkar, Tim Ruffing, and Jonas Nick. Bulletproofs++: Next generation confidential transactions via reciprocal set membership arguments. Cryptology ePrint Archive, Report 2022/510, 2023. Revised July 2023. https://eprint.iacr.org/2022/510.

[Por13] Thomas Pornin. Deterministic usage of the Digital Signature Algorithm (DSA) and Elliptic Curve Digital Signature Algorithm (ECDSA). RFC 6979, August 2013. https://www.rfc-editor.org/rfc/rfc6979.

[Sch91] Claus-Peter Schnorr. Efficient signature generation by smart cards. *Journal of Cryptology*, 4(3):161–174, 1991.